

KLDivLoss#

<https://pytorch.org/docs/stable/generated/torch.nn.KLDivLoss.html#torch.nn.KLDivLoss>

Description:

The Kullback-Leibler divergence loss. For tensors of the same shape y_{pred} , y_{true} , where y_{pred} is the input and y_{true} is the target, we define the pointwise KL-divergence as $\sum_i y_{\text{pred},i} \log \frac{y_{\text{pred},i}}{y_{\text{true},i}}$. To avoid underflow issues when computing this quantity, this loss expects the argument input in the log-space. The argument target may also be provided in the log-space if `log_target= True`. To summarise, this function is roughly equivalent to computing $\sum_i y_{\text{pred},i} \log \frac{y_{\text{pred},i}}{y_{\text{true},i}}$ and then reducing this result depending on the argument reduction as

Function Signature

```
class torch.nn.KLDivLoss(size_average=None, reduce=None,
reduction='mean', log_target=False)[source]#
```

Parameters

Shape:

```
forward(input, target)[source]#
```

Return type

Returns:

Tensor

Code Examples

```
if not log_target: # default
    loss_pointwise = target * (target.log() - input)
else:
    loss_pointwise = target.exp() * (target - input)
```

```
if reduction == "mean": # default
    loss = loss_pointwise.mean()
elif reduction == "batchmean": # mathematically correct
    loss = loss_pointwise.sum() / input.size(0)
elif reduction == "sum":
    loss = loss_pointwise.sum()
else: # reduction == "none"
    loss = loss_pointwise
```

```
>>> kl_loss = nn.KLDivLoss(reduction="batchmean")
>>> # input should be a distribution in the log space
>>> input = F.log_softmax(torch.randn(3, 5, requires_grad=True),
dim=1)
>>> # Sample a batch of distributions. Usually this would come
from the dataset
>>> target = F.softmax(torch.rand(3, 5), dim=1)
>>> output = kl_loss(input, target)
>>>
>>> kl_loss = nn.KLDivLoss(reduction="batchmean",
log_target=True)
>>> log_target = F.log_softmax(torch.rand(3, 5), dim=1)
>>> output = kl_loss(input, log_target)
```

Notes & Warnings

NOTE

Note As all the other losses in PyTorch, this function expects the first argument, input, to be the output of the model (e.g. the neural network) and the second, target, to be the observations in the dataset. This differs from the standard mathematical notation $KL(P \parallel Q)$ where P denotes the distribution of the observations and Q denotes the model.

⚠ WARNING

Warning reduction= “mean” doesn’t return the true KL divergence value, please use reduction= “batchmean” which aligns with the mathematical definition.

Detailed Documentation

Documentation

The Kullback-Leibler divergence loss.

For tensors of the same shape y_{pred} , y_{true} , where y_{pred} is the input and y_{true} is the target, we define the pointwise KL-divergence as

To avoid underflow issues when computing this quantity, this loss expects the argument input in the log-space. The argument target may also be provided in the log-

space if `log_target= True`.

To summarise, this function is roughly equivalent to computing

and then reducing this result depending on the argument `reduction` as

As all the other losses in PyTorch, this function expects the first argument, `input`, to be the output of the model (e.g. the neural network) and the second, `target`, to be the observations in the dataset. This differs from the standard mathematical notation $KL(P \parallel Q)$ where P denotes the distribution of the observations and Q denotes the model.

`reduction= "mean"` doesn't return the true KL divergence value, please use `reduction= "batchmean"` which aligns with the mathematical definition.

`size_average` (bool, optional) – Deprecated (see `reduction`). By default, the losses are averaged over each loss element in the batch. Note that for some losses, there are multiple elements per sample. If the field `size_average` is set to `False`, the losses are instead summed for each minibatch. Ignored when `reduce` is `False`. Default: `True`

`reduce` (bool, optional) – Deprecated (see `reduction`). By default, the losses are averaged or summed over observations for each minibatch depending on `size_average`. When `reduce` is `False`, returns a loss per batch element instead and ignores `size_average`. Default: `True`

`reduction` (str, optional) – Specifies the reduction to apply to the output. Default: `"mean"`

`log_target` (bool, optional) – Specifies whether target is the log space. Default: `False`

Input: $(*)^{(*)} (*)$, where $*$ means any number of dimensions.

Target: $(*)^{(*)} (*)$, same shape as the input.

Output: scalar by default. If reduction is 'none', then $(*)^{(*)} (*)$, same shape as the input.

Runs the forward pass.